Foundation for Virtual and Augmented Reality Systems

Prof. Samit Bhattacharya

Department of Computer Science and Engineering

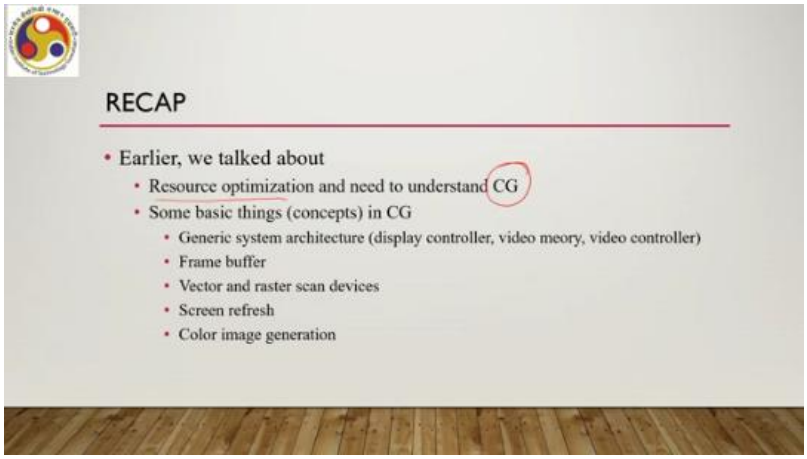
Indian Institute of Technology, Guwahati

Week – 11

Lecture – 55

VR resource optimization

Hello and welcome to the NPTEL MOOCs course on Foundation for Virtual and Augmented Reality Systems. We are going to continue our discussion on resource optimization. So, earlier we talked about the basic idea of optimization, resource optimization, and in order to fully understand the resource optimization issue and solutions, we also talked about the need to understand computer graphics or CG, which basically deals with how to render an image on the display. Now, we also briefly covered some basic concepts about computer graphics, namely the idea of a generic system architecture.



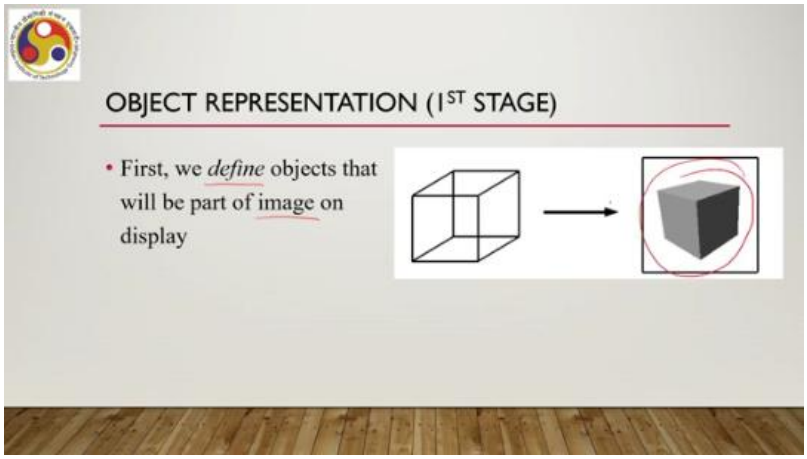
So, in a very broad sense, how a computer graphics system can be organized. We talked about three major components: display controller, video memory, and video controller. Then we talked about frame buffer, vector and raster scan devices, the idea of screen refreshing, and color image generation. So, these are very basic concepts and terminologies that we shall be referring to in our subsequent discussions. So, it is important that you remember these terms and the concepts associated with them. Now, in this lecture, we are going to introduce the 3D graphics pipeline, and we are also going to talk about parts of the pipeline in this part of the lecture. So, let us first talk about the 3D graphics pipeline: what it is and why we need to know it. Earlier, we talked about color values that are stored in the frame buffer.

So, based on these values, the video controller controls the pixels so that we perceive a colored image on the screen. The question is, How are these values obtained? From where do we get these values? The display processor, which is part of the display controller, is responsible for computing these values in stages.

So, there are a few important terms here. One is, that we have some values, real numbers, stored in the framebuffer. These values are used by the video controller to drive the display mechanism, that is, the excitation of pixels, and the values are computed by the display processor, which is part of the display controller unit, and these computations take place in stages, distinct stages. Now, these stages together are known as the graphics pipeline. Since here we are dealing with 3D scenes, we will be talking about the 3D graphics pipeline. So, essentially the process, the stage-wise process to compute the values stored in the frame buffer, which is used to drive the display mechanism by the video controller, that process, the stage-wise process, is the 3D graphics pipeline.

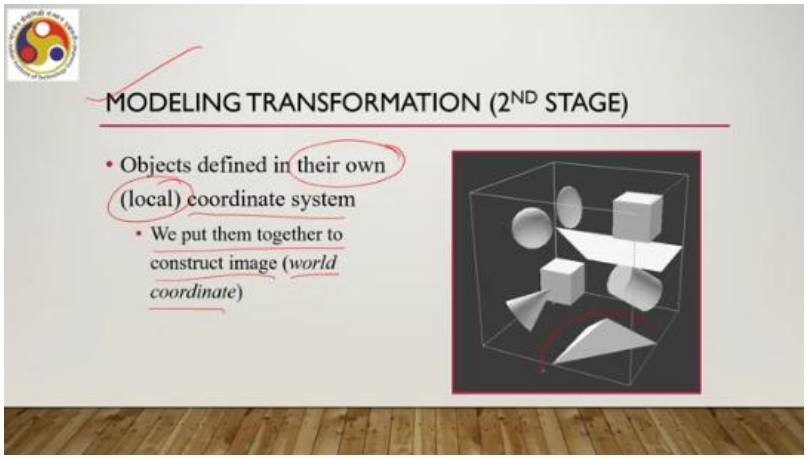
So let us have a basic understanding of what happens in this pipeline. First, we define objects that will be part of the image on display. So if you recollect our previous discussion, we mentioned that when we see something on a screen, it comprises objects which may or may not include interface level objects. However, the idea is that first we need to define these objects that together constitute the image that is rendered on a screen.

For example, suppose I want to show this cube on the screen. So, then I need to define this cube somewhere. So that is the first stage, that is representing the object. So from the representation, the final rendering takes place. Then in the second stage, we perform something called modeling transformation.

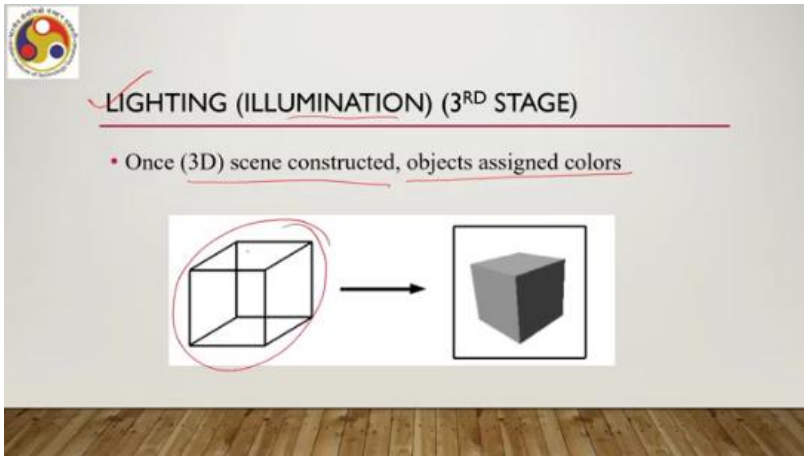


When we are saying that we define the object, essentially what we do is we are defining the objects in their own coordinate system which is often called the local coordinate system, local to the object definition. Now, when we are trying to render a scene, the scene is made up of multiple objects, each object defined in its own coordinate. So, we need to put them together in a common coordinate system to construct the overall image. So, essentially we put them together in a common coordinate system to construct the image that is called the world coordinate system.

So there is a transformation from the local coordinate system to the world coordinate system, and that is the second stage: the modeling transformation. Like here, so each of these individual objects that you can see here is defined in its own coordinate system. Multiple objects are there of different shapes, sizes, orientations, everything. Now, to construct a scene, we are putting them together. And while doing so, we are essentially bringing them together in a common coordinate system applicable to all objects, and that coordinate system is the world coordinate system.



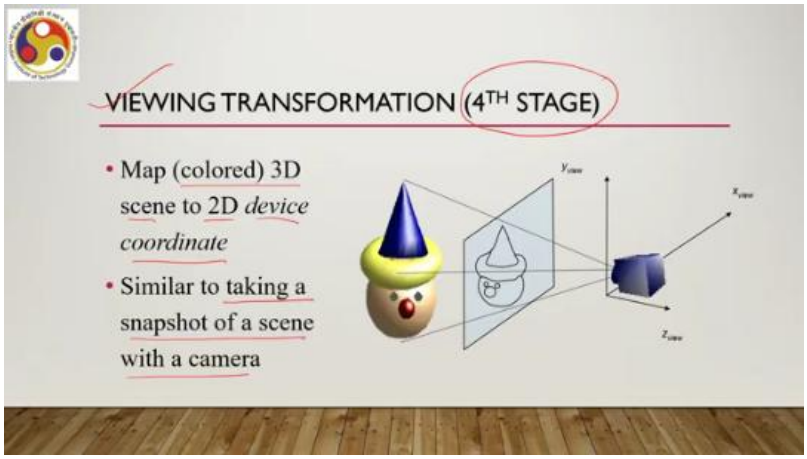
So, modeling transformation transforms objects from their local coordinate system definition to objects in the world coordinate system definition. The third stage is the lighting stage, the lighting or illumination stage. Once the 3D scene is constructed by putting the objects together, we can assign color to the objects. Say here, this is the object definition. It is barely the boundaries shown here.



However, when we assign color, we get something like this. As you can see, without assigning color, we do not perceive the 3D nature of the scene. It is there, but once we assign color, it becomes more prominent. The perception becomes better. So, essentially, assigning color is very important, and that we do in a separate stage called the lighting or illumination stage.

In our pipeline, we will be doing this as the third stage of the pipeline. Then, there is a fourth stage called viewing transformation, which maps the colored 3D scene to 2D device coordinates. So, essentially, what happens is that we try to simulate the process of taking a snapshot of a scene with a camera. When we look at a scene with our eyes, that is in a 3D world coordinate system.

Now, when we are looking at it through the camera eyepiece, that is, we are looking at it through a different mechanism, and then when we take a snapshot, that is on a 2D plane we are seeing it. So, essentially, transforming from the 3D world coordinate scene to a 2D space, which we are calling the device space or device coordinate system. The idea is shown here. Suppose this is the camera. This is the object in the world coordinate system or the scene in the world coordinate system.

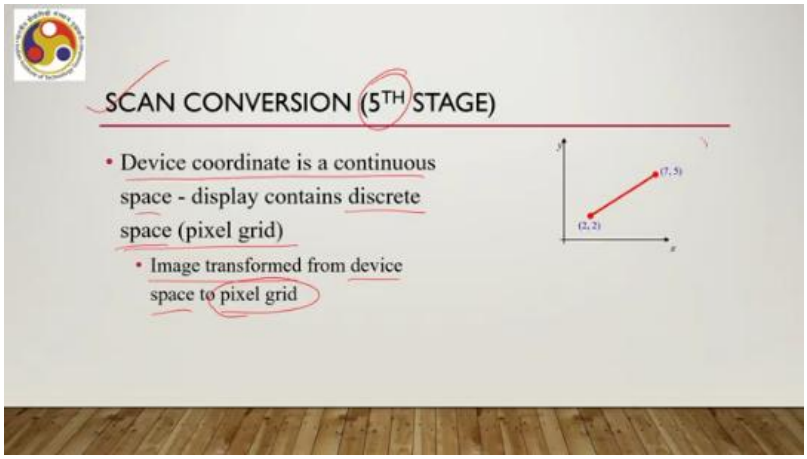


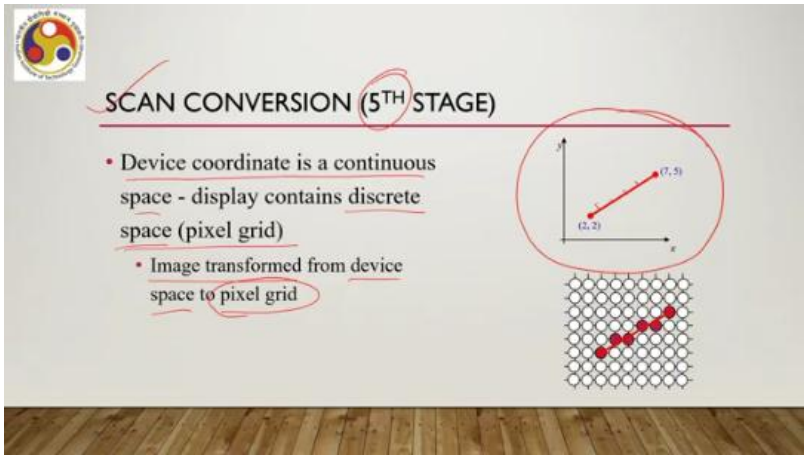
Now, this is the device coordinate system, that is, the equivalent to the screen. Note the term equivalent. It is not the same as the screen, as we shall see later, but it is the equivalent. Now, with the camera, we are taking a snapshot, meaning we are viewing this same 3D scene on a 2D plane as shown here. And this is the viewing transformation, transferring from the 3D world coordinate scene to the 2D device coordinate space.

Then there is the final or the fifth stage, which is scan conversion. Now, we said that we are through view transformation; we are actually transferring from a 3D world coordinate scene to a 2D device coordinate space. Although we are using the term device coordinate, in reality, when we talk of rendering something on a computer display, we know that the display consists of a pixel grid. Now, a pixel grid means there are discrete points. So, essentially, the device coordinate ideally should be a discrete space.

Whereas, in our earlier projection, we have assumed earlier transformation we have assumed that from a 3D world coordinate scene we are transforming to a device coordinate space which is a continuous space. That means any point can take any real number as its coordinate. However, when we are actually rendering it on a pixel grid, we know that only integer coordinates are allowed because it is a discrete space. So, essentially, what we need to do is transform the device coordinate space description to a discrete space or the pixel grid description; that is the fifth stage or scan conversion.

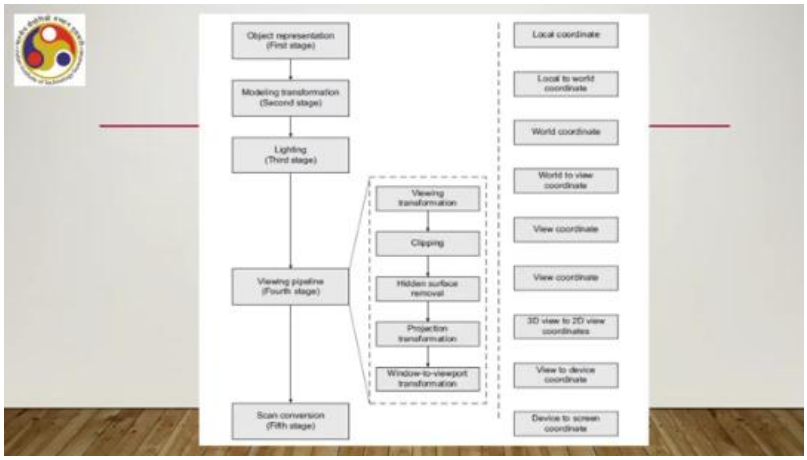
So, the image is transformed here from the device space to the pixel grid. So, suppose this is the device space description of the image, that is, a straight line as you can see. Now, intermediate points can take any real number in this device space because there is no restriction on the kind of values that the points can take. However, when we are talking of a pixel grid as shown here, we know that pixels, each pixel coordinate is represented by an integer.





It is a discrete space. So, points on the line must be mapped to pixels. In other words, the real values of the points, that is, the coordinates of the points, must be mapped to integer values represented by the pixels. This is the scan conversion or the fifth stage of the 3D graphics pipeline.

So, putting everything together, we can say that this 3D graphics pipeline contains five stages: object representation, first stage; then modeling transformation, second stage; lighting, third stage. Now, the fourth stage, although we said view transformation, essentially it is a kind of pipeline itself that comprises multiple stages. So, let us call it the viewing pipeline, that is the fourth stage, and finally, scan conversion or the fifth stage. The viewing pipeline comprises several sub-stages, such as viewing transformation. So, we will go through each of these stages in brief to get the idea.



Then there is a process called clipping. Another process called hidden surface removal, then one more transformation, projection transformation, and finally, window-to-viewport transformation. So, together, all these stages basically transform a scene in the world coordinate system to a scene in the device coordinate system. Now, the coordinate system on which these processes work, or the transformation that takes place, is also shown on the right-hand side.

So, object representation happens in the local coordinate system, local to the objects. Then modeling transformation is essentially transformation from local to world coordinate system. Lighting happens in the world coordinate system. Then viewing transformation essentially transforms from world to view coordinate system.

This is a new coordinate system which we will be talking about later. Clipping happens in the view coordinate system. Hidden surface removal and other processes happen in the same view coordinate system. Then projection transformation essentially projects or transforms 3D view coordinate system description of the scene to a 2D view coordinate system description of the scene.

Then window to viewport transformation essentially maps the 2D view coordinate or view coordinate system to the device coordinate system. So, this one is 2D view coordinate system to the device coordinate system. And finally, scan conversion or the fifth stage essentially transforms from the device coordinate system to the screen or pixel screen coordinate system or the pixel grid. So, if you notice, in order to perform these five stages, we deal with several coordinate systems.

So, essentially one is the local coordinate system, then the world coordinate system, then the 3D view coordinate system. These are all 3D view coordinate systems. Then, the 3D view to 2D view coordinate transformation means another view coordinate system that is the 2D view coordinate system. Then, the device coordinate system and, finally, the screen coordinate system. So, essentially, seven coordinate systems are involved in this overall process.

Again, just to recap: one is the local, then world, then 3D view. Then, the 2D view coordinate system, then the device coordinate, and finally, the pixel grid or screen coordinate system, so not seven. So, there are six coordinate systems that are used for

transforming from the object definition to the pixel-level description of the scene. Now, when I say pixel-level description, essentially this means that through all these stages, the intensity values for the pixels are calculated.

So, essentially, these stages represent the computation involved in calculating the intensities for each pixel, which will be stored in the frame buffer and which will be used to drive the video controller mechanism. Now, that is the overall 3D graphics pipeline. So, let us quickly have a look at the individual pipeline stages. We will be restricting ourselves to a very broad understanding.

overview of the stages rather than going into the intricate details of each of these stages. For further details, you may refer to the references mentioned at the end of these lectures. So, let us start with the object representation. So, when we are talking of computer generated scenes or synthetic scenes, we are dealing with objects of different shapes and sizes.

A variety of objects may be there. It can range from tiny snowflakes, for example, to complex animated characters. Now, if we are talking of a snowflake, it is a quite elegant structure. An elegant shape is there, and ideally it should not be represented with a simple sphere. Then the elegancy, the intricate details of the shape will be lost.

It will be a very crude approximation of the shape, and then the realism will be affected. Similarly, when we are talking of an animated character, a complex character, it needs to be depicted with its associated complexity so that it looks realistic. Again, we should not go for a very simplistic approximation, then the whole purpose of having a realistic viewing experience will be lost. So, what these examples illustrate is that we have a variety of objects; we need to render a variety of objects with their intricate details, elegant shapes, sizes, and characteristics. Now, to generate scenes with all these disparate objects, we need some way to first represent them.

And any representation will not work. So, if we are using simplistic geometric shapes to represent all the objects, certainly that will lead to a loss of reality. It will not be able to approximate all intricate characteristics of the shapes. So, any representation will not work. So, there are two fundamental questions.

One is how we can represent different objects with their characteristic complexities so that these can be rendered realistically in a synthesized environment. So, essentially we wish to capture all the intricate details of the object so that we can realistically render them. That is the first question. So, how to do that?

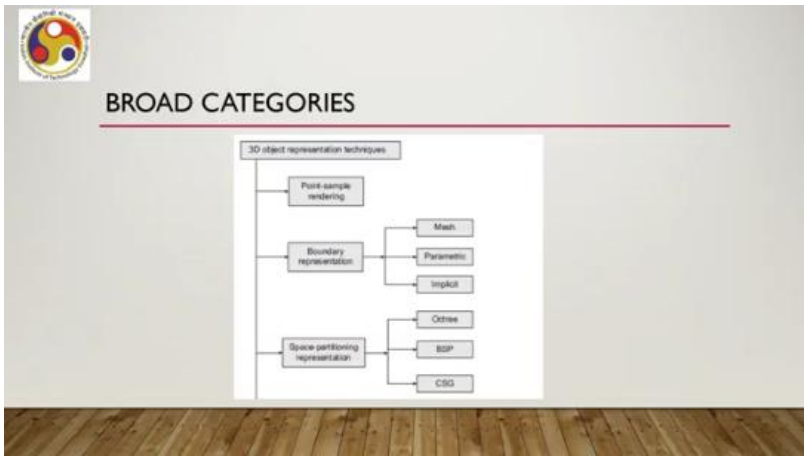
The second one is, how can we have a representation that makes the process of rendering efficient? In other words, can we perform the operations of the different stages of the pipeline that we have just discussed, the 3D graphics pipeline, in ways that optimize space and time complexities. So, when we say that through the 3D graphics pipeline stages we compute the intensities at pixel levels, these computations depend on the representation. So, the more complex the representation is, the more complex the calculations will be.

So, essentially our objective is to have a balance. So, the second fundamental question is how we can maintain that balance. We have, on one hand, a representation that is able to

represent all intricate details. On the other hand, we want to perform the subsequent stages efficiently so that the overall computation is efficient. Otherwise, realistic rendering will be again an issue.

There will be lags and other things, or we will be requiring expensive hardware. for doing the calculation. So, one question is to be able to capture all characteristics; the second question is to be able to capture them so that subsequent computations are efficiently done by balancing some trade-off. So, these two fundamental questions are there when we talk of representing objects. Now, the solution lies

in the fact that we cannot rely on a single representation technique. We have to rely on different object representation techniques. In fact, there is a plethora of such techniques available, and we have to choose the right technique to represent an object given a particular problem. Now, let us quickly focus on the broad categories of such representation techniques. So, if we are talking of 3D object representation techniques, the broad categories are three: point sample rendering, boundary representation, and space partitioning based representation.



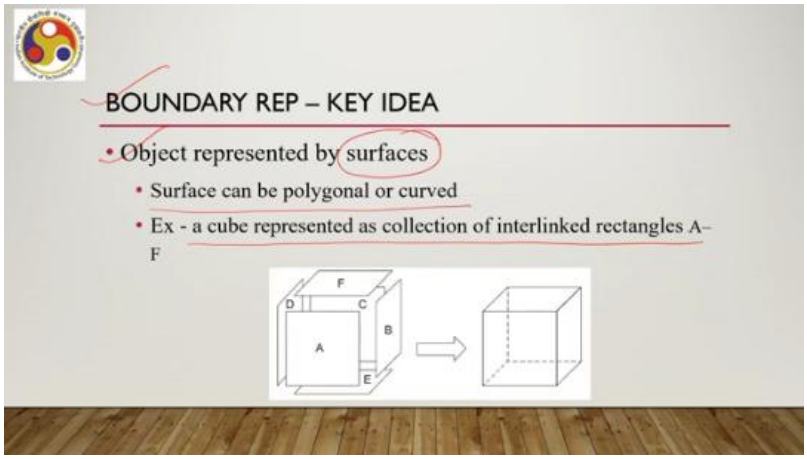
Under boundary representation, we have mesh representation, parametric representation, and implicit representation. Under space partitioning, we have the octree method. Octree method of representation, binary space partitioning or BSP method of representation, and constructive solid geometry or CSG method of representation. Note that we will not go into the details of individual techniques that are mentioned here. Again, please refer to the reference mentioned at the end.

However, let us get some quick idea on each of these broader categories: that is, point sample rendering, boundary representation, and space partitioning-based representation. So, point sample rendering is the simplest of all. Here, the information captured is in the form of points on the scene, with each point having the information as attributes. Now, the capturing of such information happens with input sensors such as 3D range scanner, range finder, or computer vision techniques. So, essentially, what we do here is to generate a scene; we first capture the scene from the real world by some input sensors. The capturing essentially takes place in the form of capturing information related to points on the scene, and those points are subsequently mapped to the pixels. So, these points can subsequently

be processed or mapped to the pixels to generate the scene. So, here we are relying on the real-world information to generate a synthetic image.

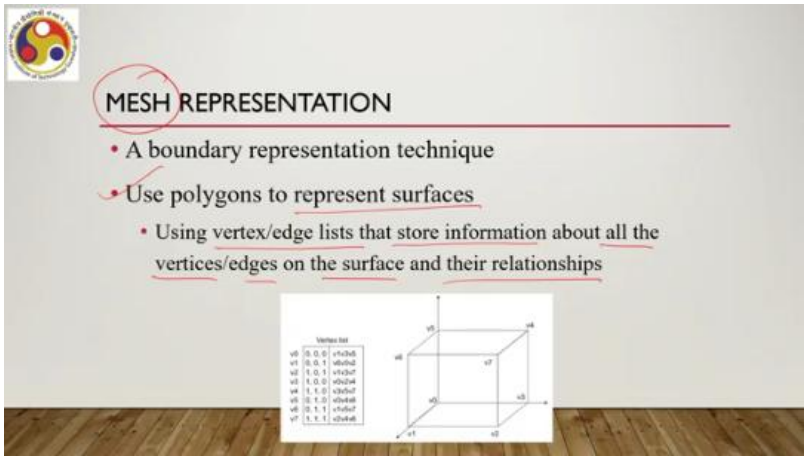
We are not synthetically defining objects here. So, in this case, the representation is nothing but a set of raw data points. That is the point sample-based rendering. Next is the boundary representation, the second broad category of representation. Here, the objects are represented by the surfaces that define the boundary of the object.

Now, these surfaces can be polygonal or curved surfaces. For example, a cube represented as a collection of internally linked rectangles is shown here. So, the rectangles are numbered or identified with letters between A to F. So, this is one rectangle, second rectangle, third rectangle, fourth, fifth, sixth. So, six rectangles each representing the bounding surface of a cube. Now, when we are representing this whole cube in the form of its bounding surfaces, then we are saying that we are following a



boundary representation. Now, one very popular boundary representation technique is called mesh representation. In this mesh representation, we use polygons to represent the surfaces, such as the one shown in the previous example. Now, in order to represent each polygon, what we use is basically a vertex or edge list. That stores information about all the vertices or edges on the surface and their relationships, how the vertices are connected to each other or edges are connected to each other.

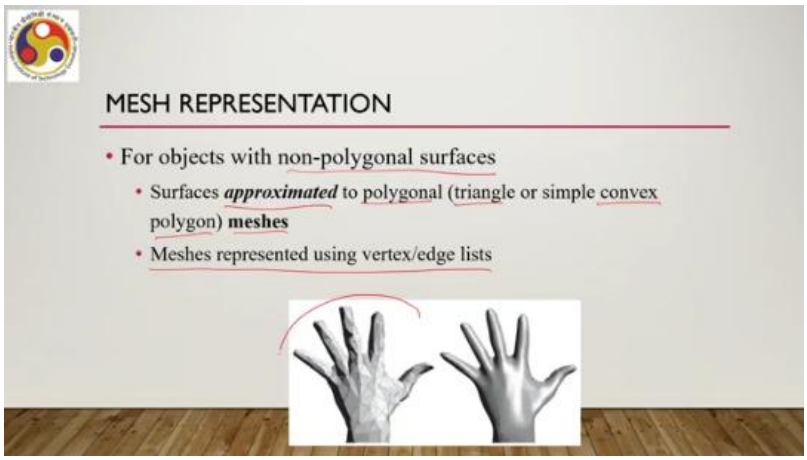
For example, this cube we can represent in the form of a vertex list as shown here. Say V0 is the origin; then it says that it is connected to V1. V3 and V5. So, this entry indicates the coordinate of the vertex and which other vertices it is connected to. Similarly, V1, V2, all 8 vertices are mentioned in the list along with their connections to other vertices.



From here, we can reconstruct the cube if the information is captured in this way. So, here what we see is that essentially it refers to a list of vertices, and these vertices are related to the bounding surfaces which define the overall object. So, this is a kind of boundary representation; this is known as the mesh representation because here we are assuming that the bounding surfaces are polygons, and polygons are connected together to create a mesh which represents the whole object.

So, if there are objects with non-polygonal surfaces, then can we represent them using the mesh representation? The answer is yes. We can approximate the surfaces to polygonal meshes. Now, typically, meshes are constructed using triangles.

However, other simple convex polygons may also be used, and the meshes are represented using vertex or edge lists as shown before. So, for example, for this hand here, there is no intuitive way to create a polygonal mesh to represent the hand because the surfaces are curved surfaces. However, as shown in this figure, we can use a triangular mesh with all these triangles, as you can see, to represent this hand and then subsequently process this mesh to represent or reconstruct the hand here or render it on the screen. Clearly, as you can see or as is evident from this image,



the greater the number of polygons in the mesh, the better the approximation it will lead to. However, the greater the number of polygons in the mesh, the more complex the vertex

list which represents the mesh will be. So, constructing that vertex list will be even more difficult with the increase in the number of polygons. So, again here, there is a balance we have to maintain; there is a trade-off we have to address.

Now, this mesh is the most basic form of representation for any object in a rendered scene used in computer graphics. So, no matter what kind of representation we are following, eventually all of them are mapped to a mesh representation. So, all other representations are converted to a mesh representation at the end of the pipeline before the objects are rendered. Another way of representing curved surfaces or curves is the spline representation.

So, this is a higher-level representation typically preferred by developers or designers of scenes who want to create an artificial environment. So, splines can be used to construct quality scenes for rendering, and splines are required to represent complex surfaces, surfaces with complex properties so that those cannot be represented with simple polygons or simple polynomial equations. Now, this curve representation using splines falls under the broad category of parametric representation.

So, typically what happens is that when we try to represent a curve, we can use a polynomial to represent the curve. The more complex the curve is, the degree of the polynomial increases. Now, as the degree of the polynomial increases, computing any point on the curve using a very complex or higher-degree polynomial becomes more and more time-consuming from the point of view of computation. So, essentially with spline representation, the idea that is followed is that instead of representing a curve using a single higher-degree complex polynomial,

we can represent it using several polynomials, each with a lower degree to a manageable level. So, the complete curve consists of several pieces. Now, each piece is called a blending function or a basis function, and all pieces are of lower order so that simpler polynomials can be used to compute points on the segment of the curve represented by that particular polynomial piece. And third order is the most common form of polynomial used to construct spline curves.

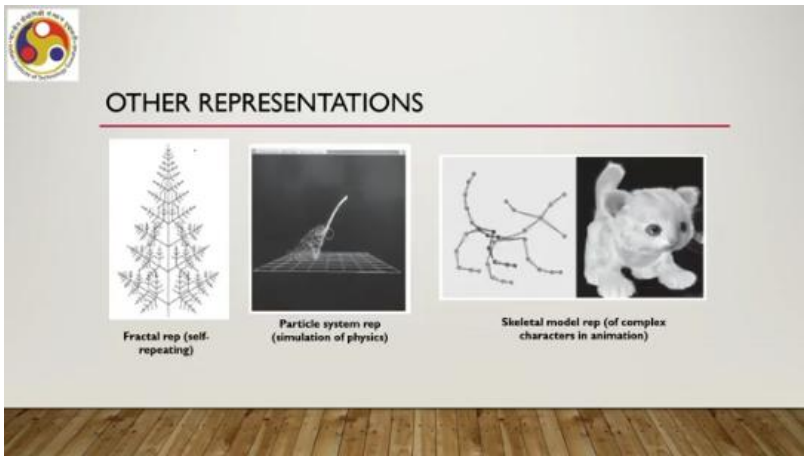
We also have to ensure that the way we define the polynomial pieces, they must join smoothly, for that certain geometric and other properties need to be satisfied by each of these pieces, but those details we can skip here. So, that is about boundary representation. Mesh is the most basic form of boundary representation. However, higher-level boundary representations using implicit or parametric forms are also there. One such

representation is the spline representation. Then there is the third broad category, that is space partitioning representation. The key idea here is that the 3D space occupied by the object is divided into several disjoint regions, disjoint or non-overlapping regions. Any point inside the object lies in exactly one of the regions. And this way we can represent the object by representing the region occupied by the object.

Now, in order to create this particular representation, we need to create a hierarchical way of representation which is called a space partitioning tree. So, kind of tree-like structure which represents the space occupied by the object. So, there are, as we mentioned, several

such representations including Oct tree method, binary space partitioning, or constructive solid geometric methods; again, we will skip the details. Now, apart from these broad categories, there are several techniques used to represent

objects that are application-specific or very complex photorealistic objects. For example, the structure of this leaf, as you can see here, is self-repeating; that is, each individual part is repeating in the larger part. Now, this type of elegant structure ideally requires a special way of representation. So, one such way is fractal representation.



It is a specialized way to represent self-repeating structures. Similarly, this waterfall, again, requires an elegant way of representation rather than these broader categories of boundary or space partitioning methods. So, one such representation is the particle system representation. which essentially refers to or relies on the simulation of physical properties. Then, representing this complex character, such as a cat, again requires or calls for some sophisticated way to represent it.

One is the skeletal model, as shown here on the left side of the actual figure. So, this skeletal model tries to capture the overall appearance of the character, plus the motion behavior of the character can be captured using the skeletal model. Typically, such models are used to represent objects, complex characters in animation, or computer animation. These are some of the specialized techniques; there are other techniques also.

So, broadly, although we can have some categories, there are specialized, sophisticated techniques that are often required for representing photorealistic objects. So, let us quickly summarize what we have learned so far. We have gone through, in this part of the lecture, the idea of the 3D graphics pipeline, where we talked of several stages, broadly five stages: namely, object representation, modeling transformation, viewing transformation, and scan conversion.

As we have seen, they involve some transformations from one coordinate system to another; a total of six coordinate systems are involved here. And we talked about the first stage of this pipeline, that is, object representation technique. One thing to note here is that, for the sake of discussion, we are representing the pipeline in the way we are discussing. It may be noted that this sequence may not be exactly followed when the pipeline is

implemented. So, although we say that the first stage is object representation, the second one is modeling transformation,

the third one is lighting, the fourth one is viewing transformation, and the fifth is scan conversion. It need not be the exact same sequence when an implementation of this 3D pipeline is done. Stages may change; substages may change their order from the way we are discussing here. The whole purpose of discussing in this particular way is to give some idea of how the computations take place in a stage-wise manner. It may be noted that the sequence is not that important, but the stages are very important.

So, all stages will be there. but they may follow a slightly different sequence than the way we are discussing here. Now, we will continue our discussion on these pipeline stages in the subsequent parts of the lecture as well. I hope you could follow the discussion and understood the topics covered. Here we are discussing everything in a very brief manner to give you an overall idea, as a detailed discussion of computer graphics is not in the scope of this particular course.

So, looking forward to meeting you again in the next part of the lecture. Till then, thank you, goodbye, and happy learning.